

Chapter 7 - Unified Modelling Language (UML)

Static Diagrams (Structural)

1. Use Case
2. Class

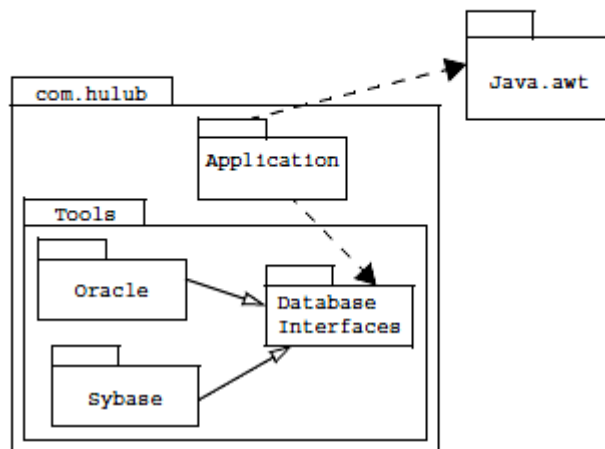
Dynamic Diagrams (Behavioral)

1. Object
2. Sequence (Interaction)
3. Collaboration (Interaction)
4. State Chart
5. Activity

Implementation Diagrams

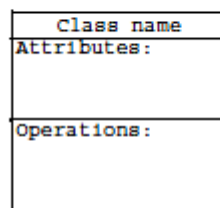
1. Component
2. Deployment

Packages



- C++ **namespace**.
- Group together functionally-similar classes.
- Derived classes need not be in the same package.
- Packages can nest. Outer packages are sometimes called **domains**. (In the diagram, “Tools” is arguably an outer package, not a domain).
- Package name is part of the class name (e.g. given the class *fred* in the flintstone package, the **fully-qualified** class name is *flintstone.fred*).
- Generally needed when entire static-model won't fit on one sheet.

Classes (Box contains three compartments)



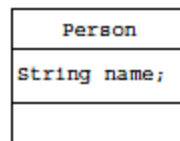
1. The name compartment (required) contains the class name and other documentation-related information:

E.g.:

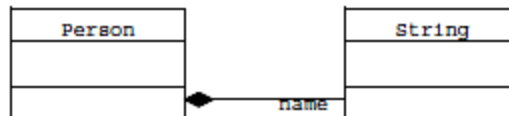
```
Some_class «abstract»  
{ author:   George Jetson  
  modified: 10/6/2999  
  checked_out: y  
}
```

- Guillemets identify stereotypes. E.g.: «static», «abstract» «JavaBean». Can use graphic instead of word.
 - Access privileges (see below) can precede name.
 - Inner (nested) classes identify outer class as prefix of class name: (**Outer.Inner** or **Outer::Inner**).
2. The *attributes compartment* (optional):
- *During Analysis*: identify the attributes (i.e. defining characteristics) of the object.
 - *During Design*: identify a relationship to a stock class.

This:



is a more compact (and less informative) version of this:



Everything, here, is private. Always. Period.

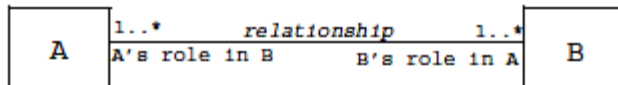
3. The *operations compartment* (optional) contains method definitions. Use implementation-language syntax, except for access privileges:

+	public
#	protected
-	private
~	package (my extension to UML) ¹

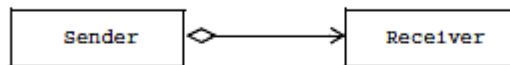
- Abstract operations (C++ virtual, Java non-final) indicated by *italics* (or underline).
- **Boldface** operation names are easier to read.

If attributes and operations are both omitted, a more complete definition is assumed to be on another sheet.

Associations (relationships between classes)



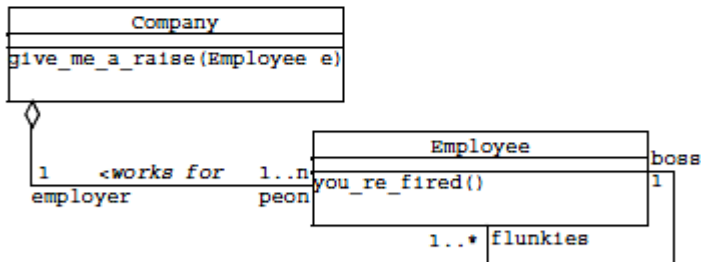
- Associated classes are connected by lines.
- The relationship is identified, if necessary, with a < or > to indicate direction (or use solid arrowheads).
- The role that a class plays in the relationship is identified on that class's side of the line.
- Stereotypes (like «friend») are appropriate.
- Unidirectional message flow can be indicated by an arrow (but is implicit in situations where there is only one role):



- Cardinality:

1	Usually omitted if 1:1
n	Unknown at compile time, but bound.
0..1	(1..2 1..n)
1..*	1 or more
*	0 or more

- Example:



```

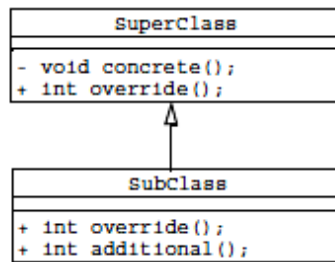
class Company
{
    private Employee[] peon = new Employee[n];
    public void give_me_a_raise( Employee e ) { ... }
}

class Employee
{
    private Company employer;
    private Employee boss;
    private Vector flunkies = new Vector();
    public void you_re_fired() { ... }
}

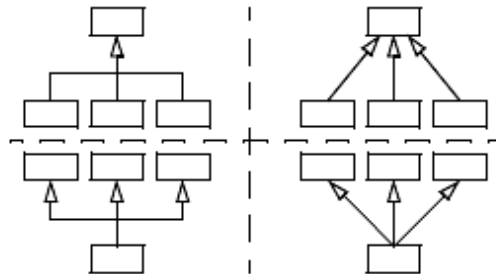
```

(A Java Vector is a variable-length array. In this case it will hold Employee objects)

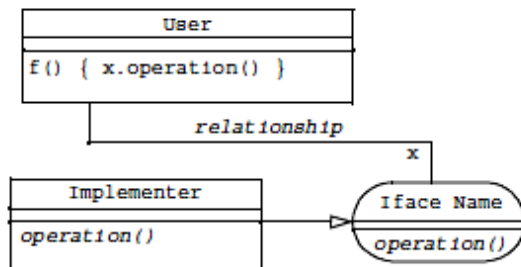
Implementation Inheritance



Outline arrows identify derivation relationships: extends, implements, is-a, has-properties-of, etc. Variations include:

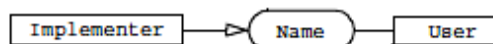


Interface Inheritance

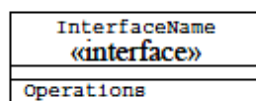


In C++, an interface is a class containing nothing but pure virtual methods. Java supports them directly (c.f. “abstract class,” which can contain method and field definitions in addition to the abstract declarations.)

My extension to UML: rounded corners identify interfaces. If the full interface specification is in some other diagram, I use:



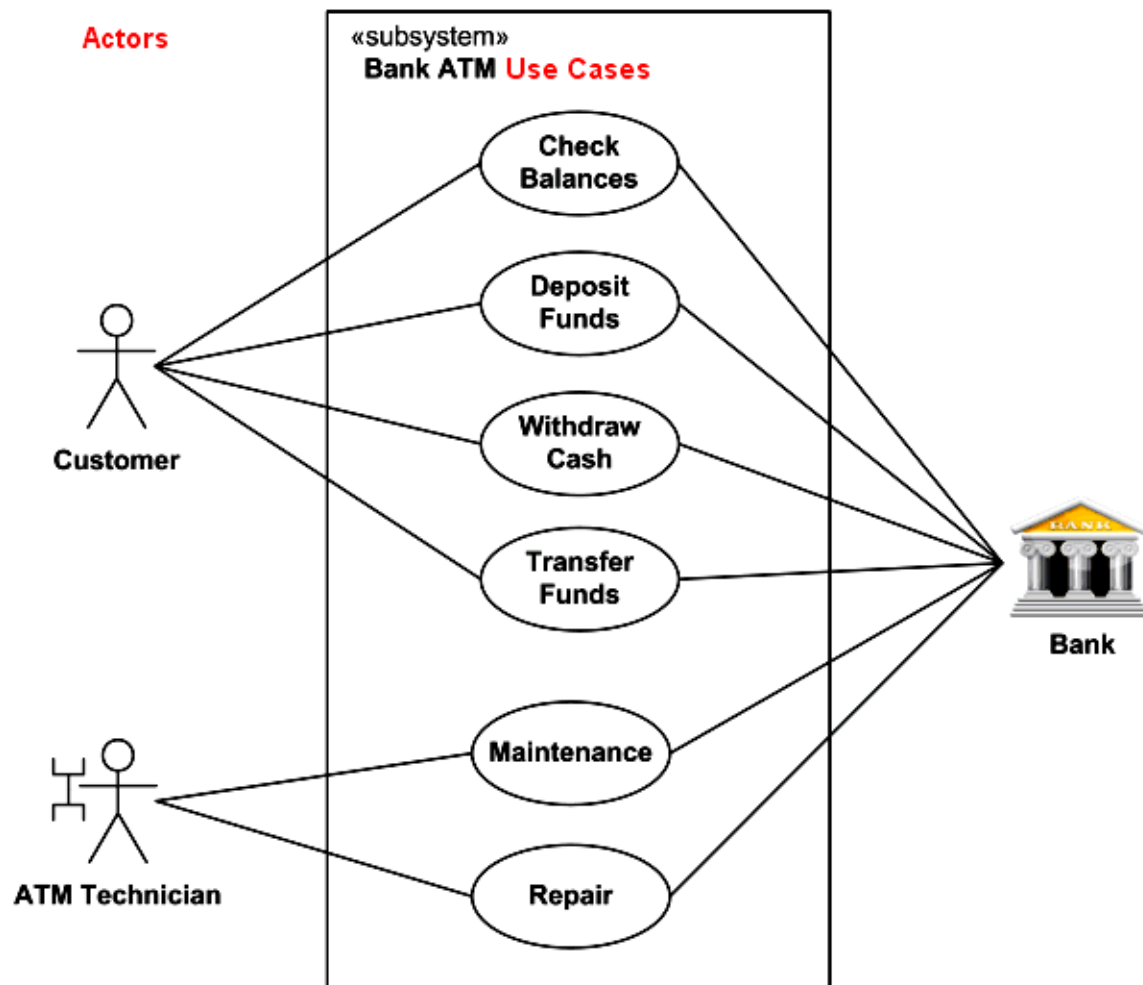
Strict UML uses the «interface» stereotype in the name compartment of a standard class box:



Interfaces contain no attributes, so the attribute compartment is always empty.

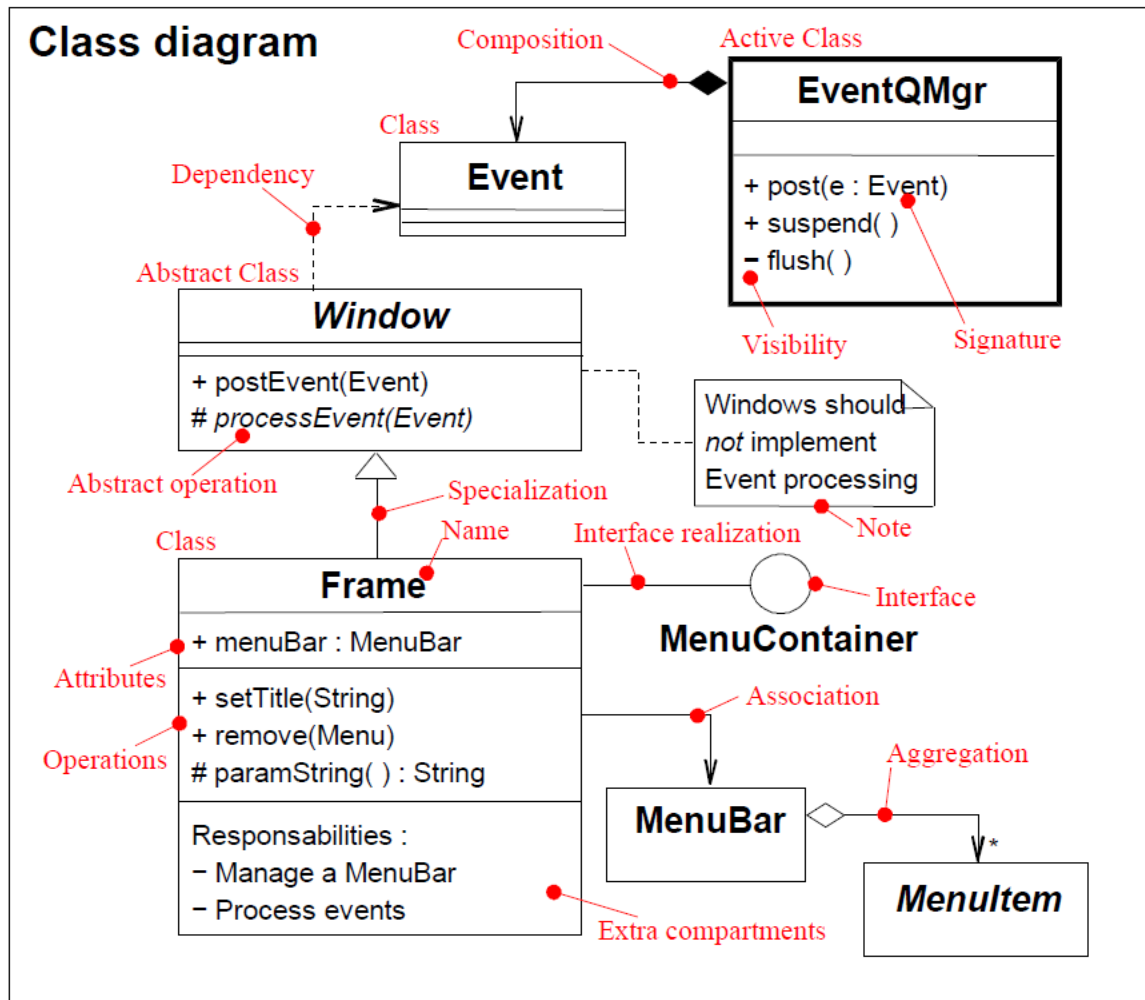
Static Diagrams – 1: Use Case (Structural)

Use case modeling is an approach to capture user requirements. A use-case is a sequence of interactions between the user and the system under consideration to achieve a goal. Use case diagrams model the functionality of a system using actors and use cases. Use cases are services or functions provided by the system to its users.



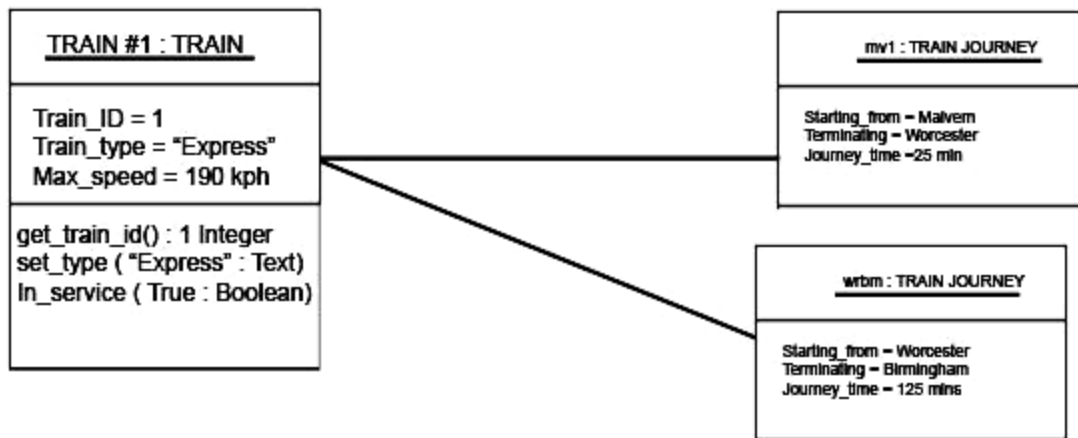
Static Diagrams – 2: Class (Structural)

A class diagram is a graphical-notation for depicting the system's classes along with their relationship to one another. Classes relate to each other through different forms of association.



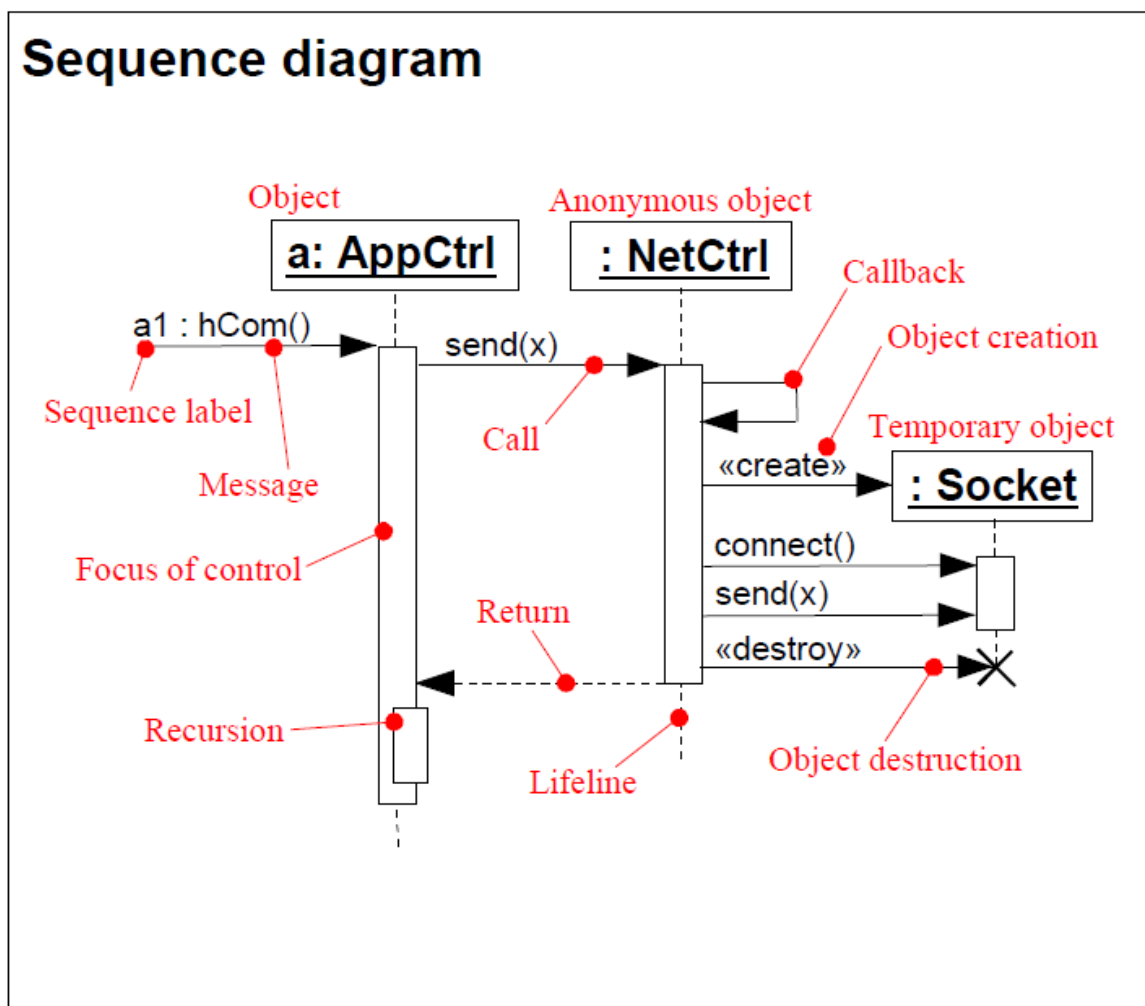
Dynamic Diagrams – 1: Object (Behavioral)

Object diagrams are also closely linked to class diagrams. Just as an object is an instance of a class, an object diagram could be viewed as an instance of a class diagram. Object diagrams describe the static structure of a system at a particular time and they are used to test the accuracy of class diagrams.



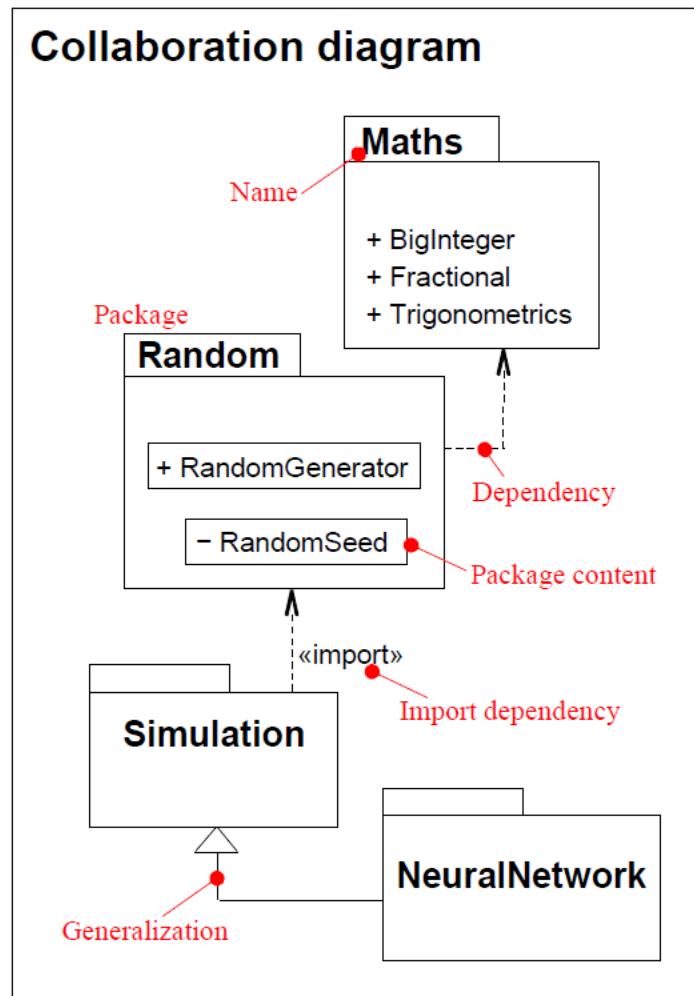
Dynamic Diagrams – 2: Sequence (Behavioral) -Interaction

Sequence diagrams describe interactions among classes in terms of an exchange of messages over time. Sequence diagrams model the dynamic aspects of a software system. The emphasis is on the “sequence” of messages rather than relationship between objects. A sequence diagram maps the flow of logic or flow of control within a usage scenario into a visual diagram enabling the software architect to both document and validate the logic during the analysis and design stages.



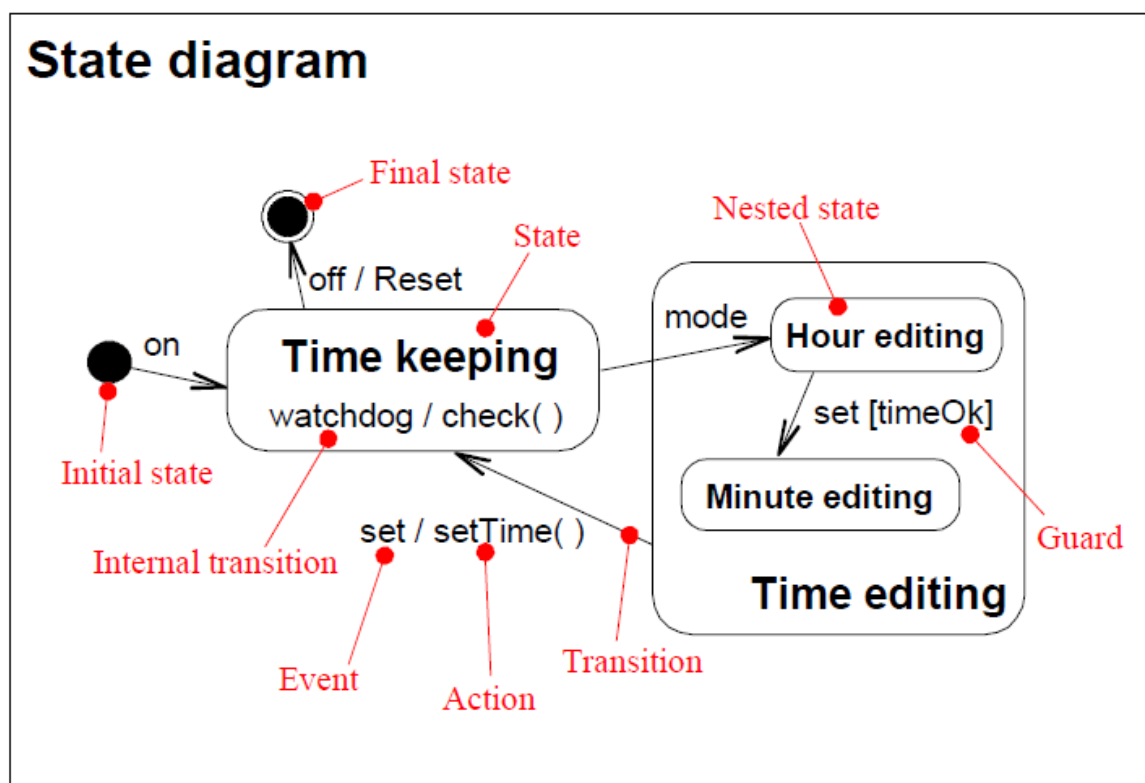
Dynamic Diagrams – 3: Collaboration (Behavioral) - Interaction

A collaboration diagram describes interactions among objects in terms of sequenced messages. Collaboration diagrams represent a combination of information taken from class, sequence, and use case diagrams describing both the static structure and dynamic behavior of a system.



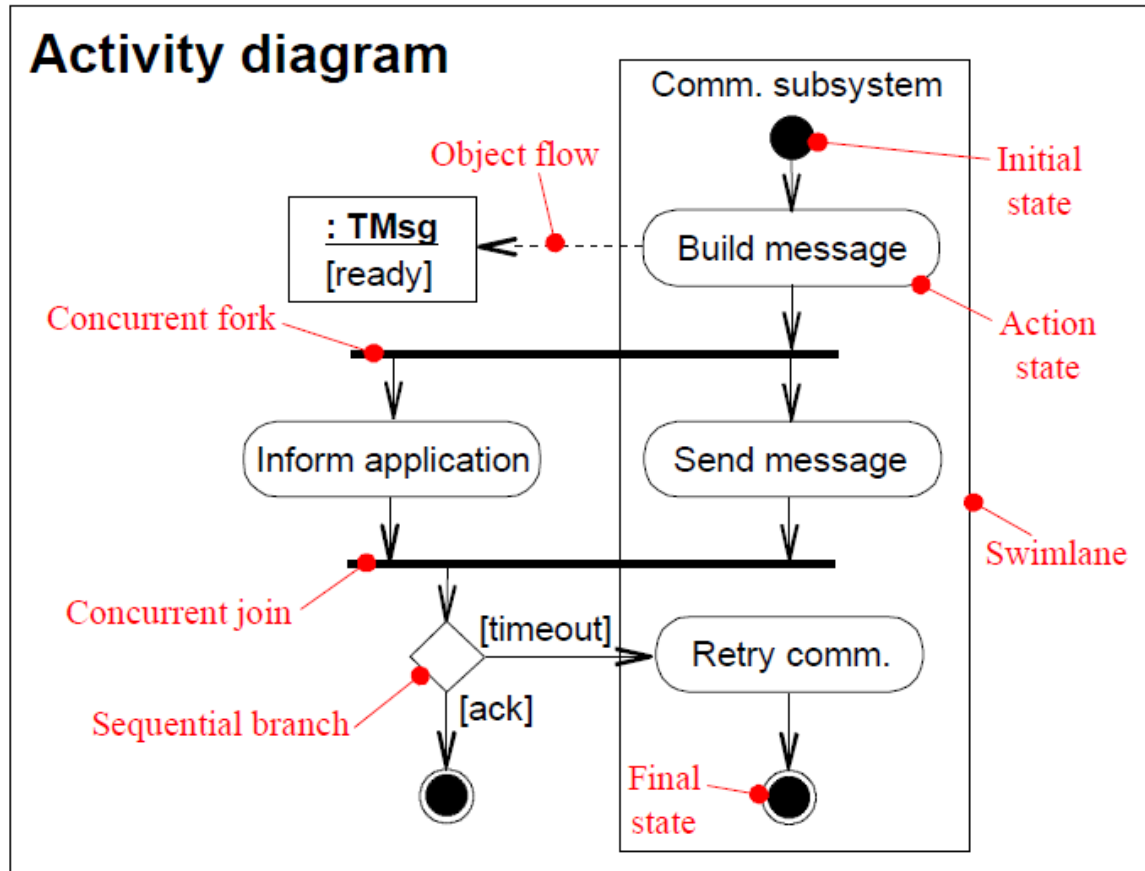
Dynamic Diagrams – 4: State Chart Diagram (Behavioral)

A state chart diagram is a graph in which nodes correspond to states and directed arcs correspond to transitions labeled with event names. **A state diagram combines states and events in the form of a network to model all possible object states during its life cycle, helping to visualize how an object responds to different stimuli.** A state chart diagram shows the behavior of classes in response to external stimuli. This diagram models the dynamic flow of control from state to state within a system.



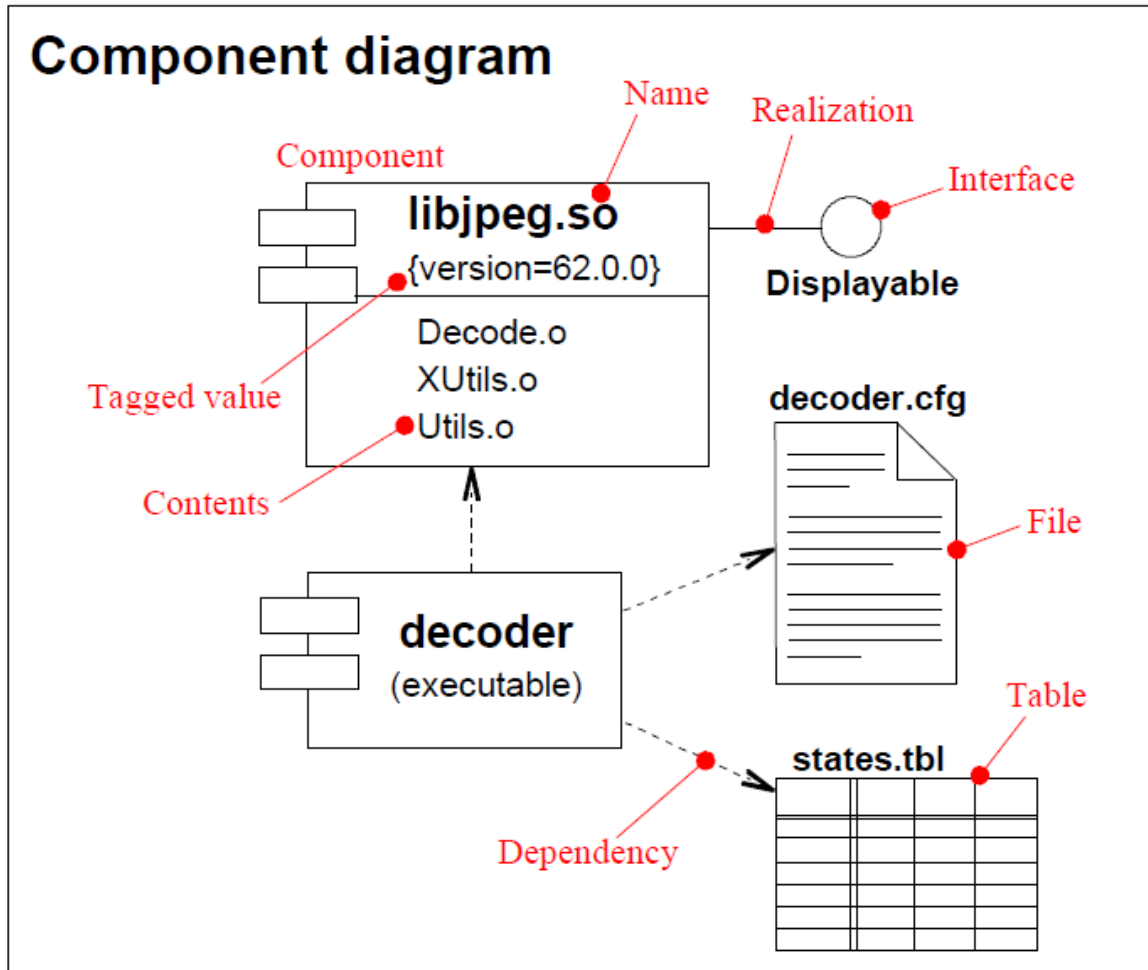
Dynamic Diagrams – 5: Activity (Behavioral)

An **activity diagram** is a type of flow chart with additional support for parallel behavior. Activity diagrams include the following new concepts. Branches and Merges model the conditional behavior of activity diagrams. A branch has a single incoming transition and multiple, conditional, outgoing transitions. The control flow is directed to one of the outgoing transitions depending on the condition satisfied. A merge is a node in the activity diagram at which the conditional behavior terminates. Each branch in an activity diagram has a corresponding merge. An activity diagram illustrates the dynamic nature of a system by modeling the flow of control from activity to activity. An activity represents an operation on some class in the system that results in a change in the state of the system. Typically, activity diagrams are used to model workflow or business processes and internal operation. Because an activity diagram is a special kind of state chart diagram, it uses some of the same modeling conventions.



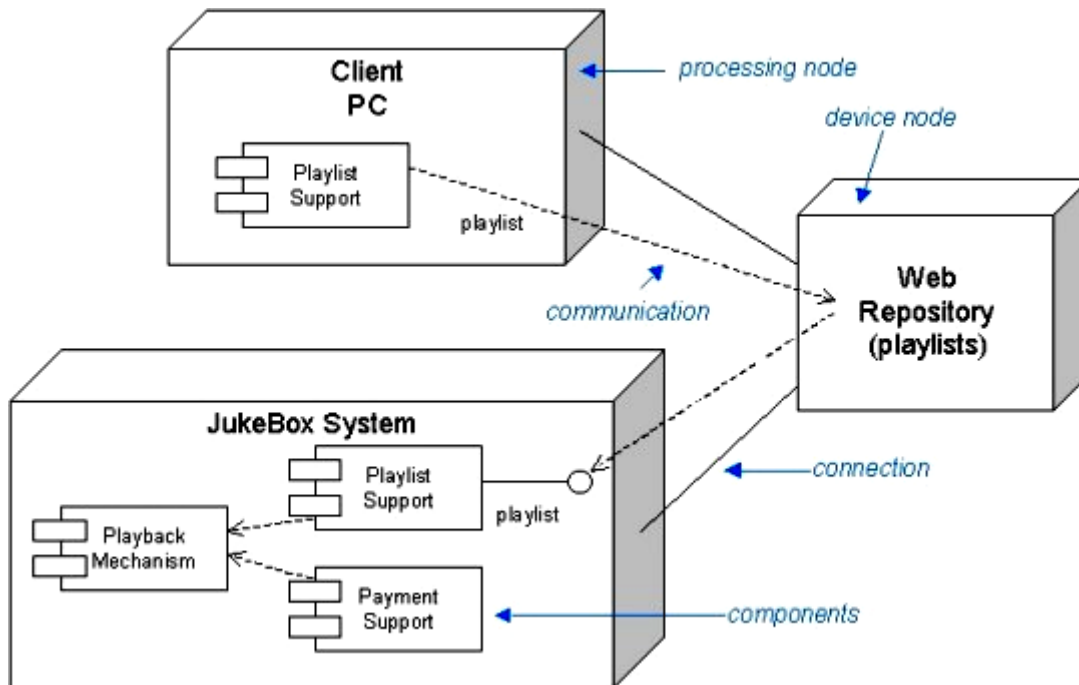
Implementation Diagrams – 1: Component Diagram

A component diagram describes the organization of the physical components in a system.



Implementation Diagrams – 2: Deployment Diagram

Deployment diagrams depict the physical resources in a system including nodes, components, and connections.



Chapter 8 - Structured Query Language (SQL)

Introduction

SQL (pronounced as Sequel), referred to as Structured Query Language is a programming language designed for managing data in relational database management systems (RDBMS).

The SQL language is subdivided into several language elements, including:

- **Clauses**, which are constituent components of statements and queries. (In some cases, these are optional.)[11]
- **Expressions**, which can produce either scalar values or tables consisting of columns and rows of data.
- **Predicates**, which specify conditions that can be evaluated to SQL three-valued logic (3VL) or Boolean (true/false/unknown) truth values and which are used to limit the effects of statements and queries, or to change program flow.
- **Queries**, which retrieve the data based on specific criteria. This is the most important element of SQL.
- **Statements**, which may have a persistent effect on schemata and data, or which may control transactions, program flow, connections, sessions, or diagnostics. SQL statements also include the semicolon (";") statement terminator. Though not required on every platform, it is defined as a standard part of the SQL grammar.
- **Insignificant whitespace** is generally ignored in SQL statements and queries, making it easier to format SQL code for readability.

Classification

The SQL statements are classified as follows:

1) Data Definition Language (DDL)

These statements are used to define the database structure or schema. It manages table and index structure. Some examples are:

1. **CREATE** - to create objects in the database
2. **ALTER** - alters the structure of the database
3. **DROP** - delete objects from the database

4. TRUNCATE - remove all records from a table, including all spaces allocated for the records are removed
5. COMMENT - add comments to the data dictionary
6. RENAME - rename an object

2) Data Manipulation Language (DML)

These statements are used for managing data within schema objects. It is the subset of SQL used to add, update and delete data. Some examples are:

1. SELECT - retrieve data from the a database
2. INSERT - insert data into a table
3. UPDATE - updates existing data within a table
4. DELETE - deletes all records from a table, the space for the records remain
5. MERGE - UPSERT operation (insert or update)
6. CALL - call a PL/SQL or Java subprogram
7. EXPLAIN PLAN - explain access path to data
8. LOCK TABLE - control concurrency

3) Data Control Language (DCL)

The Data Control Language (DCL) authorizes users and groups of users to access and manipulate data. Its two main statements are:

1. GRANT - authorizes one or more users to perform an operation or a set of operations on an object.
2. REVOKE - eliminates a grant, which may be the default grant.

4) Transaction Control Language (TCL)

These statements are used to manage the changes made by DML statements. It allows statements to be grouped together into logical transactions.

1. COMMIT - Persist the changes
2. SAVEPOINT - Identify a point in a transaction to which you can later roll back
3. ROLLBACK - Restore database to original since the last COMMIT
4. SET TRANSACTION - Change transaction options like isolation level and what rollback segment to use

Normalization – 3 Normal Forms

The purpose is to

- Remove Redundancy
- Maintain Data Integrity
- Improve Efficiency

First Normal Form (1NF) - No repeating attributes or group of attributes (No repeating columns)

Second Normal Form (2NF) - All non-key attributes must depend on the whole key (No repeating rows)

Third Normal Form (3NF) - No dependencies on non-key attributes (No transitive functional dependencies)

Cartesian product

Also referred to as a cross-join product, returns all the rows in all the tables listed in the query. Each row in the first table is paired with all the rows in the second table. This happens when there is no relationship defined between the two tables.

Joins

SQL Joins are used to relate information in different tables. A Join condition is a part of the SQL query that retrieves rows from two or more tables. A SQL Join condition is used in the SQL WHERE clause of SELECT, UPDATE and DELETE statements. SQL Joins can be classified into Equi joins and Non Equi joins.

SQL Equi Join

It is a simple sql join condition which uses the equal sign as the comparison operator. Equi Join is further classified into two categories:

- 1. SQL Inner Join**

All the rows returned by the sql query satisfy the sql join condition specified.

- 2. SQL Outer Join**

This sql join condition returns all rows from both tables which satisfy the join condition along with rows which do not satisfy the join condition from one of the tables. The sql outer join operator in Oracle is (+) and is used on one side of the join condition only. They are further classified into left and right outer joins.

Left Outer Join

If the (+) operator is used in the left side of the join condition it is equivalent to left outer join.

Right Outer Join

If used on the right side of the join condition it is equivalent to right outer join.

SQL Non-Equi Join

It is a sql join condition which makes use of some comparison operator other than the equal sign like >, <, >=, <=.

Self-Join

A Self Join is a type of sql join which is used to join a table to itself, particularly when the table has a FOREIGN KEY that references its own PRIMARY KEY. It is necessary to ensure that the join statement defines an alias for both copies of the table to avoid column ambiguity.

Visit <http://beginner-sql-tutorial.com/> and <http://www.plsql-tutorial.com/>

Oracle SQL and PL/SQL Quick Reference

SELECT Statement

```
SELECT [DISTINCT] {*, column [alias],...}
FROM table
[WHERE condition(s)]
[ORDER BY {column, exp, alias} [ASC|DESC]]
```

Cartesian Product

```
SELECT table1.*, table2.*, [...]
FROM table1, table2[, ...]
```

Equijoin (Simple joins or inner join)

```
SELECT table1.*, table2.*
FROM table1, table2
WHERE table1.column = table2.column
```

Non-Equijoins

```
SELECT table1.*, table2.*
FROM table1, table2
WHERE table1.column
BETWEEN table2.column1 AND table2.column2
```

Outer joins

```
SELECT table1.*, table2.*
FROM table1, table2
WHERE table1.column(+) = table2.column
SELECT table1.*, table2.*
FROM table1, table2
WHERE table1.column = table2.column(+)
```

Self joins

```
SELECT alias1.*, alias2.*
FROM table1 alias1, table1 alias2
WHERE alias1.column = alias2.column
```

Aggregation Selecting

```
SELECT [column,] group_function(column)
FROM table
[WHERE condition]
[GROUP BY group_by_expression]
[HAVING group_condition]
[ORDER BY column] ;
```

Group function

```
AVG([DISTINCT|ALL]n)
COUNT(*)|[DISTINCT|ALL]expr)
MAX([DISTINCT|ALL]expr)
MIN([DISTINCT|ALL]expr)
STDDEV([DISTINCT|ALL]n)
SUM([DISTINCT|ALL]n)
VARIANCE([DISTINCT|ALL]n)
```

Subquery

```
SELECT select_list
FROM table
WHERE expr operator(SELECT select_list FROM table);
single-row comparison operators
= > >= < <= <>
multiple-row comparison operators
IN ANY ALL
```

Multiple-column Subqueries

```
SELECT column, column, ...
FROM table
WHERE (column, column, ...) IN
(SELECT column, column, ...
FROM table
```

```
WHERE condition) ;
```

Manipulating Data

INSERT Statement (one row)

```
INSERT INTO table [ (column [, column...])]
VALUES (value [, value...]) ;
```

INSERT Statement with Subquery

```
INSERT INTO table [ column(, column) ]
subquery ;
```

UPDATE Statement

```
UPDATE table
SET column = value [, column = value, ...]
[WHERE condition] ;
```

Updating with Multiple-column Subquery

```
UPDATE table
SET (column, column, ...) =
(SELECT column, column, ...
FROM table
WHERE condition)
WHERE condition ;
```

Deleting Rows with DELETE Statement

```
DELETE [FROM] table
[WHERE condition] ;
```

Deleting Rows Based on Another Table

```
DELETE FROM table
WHERE column = (SELECT column
FROM table
WHERE condition) ;
```

Transaction Control Statements

```
COMMIT ;
SAVEPOINT name ;
ROLLBACK [TO SAVEPOINT name] ;
CREATE TABLE Statement
CREATE TABLE [schema.]table
(column datatype [DEFAULT expr] [...]) ;
CREATE TABLE Statement with Subquery
CREATE TABLE [schema.]table
[(column, column...)]
AS subquery
```

Datatype

VARCHAR2(size)	CHAR(size)	NUMBER(p,s)	DATE
LONG	CLOB	RAW	LONG RAW
BLOB	BFILE		

ALTER TABLE Statement (Add columns)

```
ALTER TABLE table
ADD (column datatype [DEFAULT expr]
[, column datatype]...) ;
```

Changing a column's type, size and default of a Table

```
ALTER TABLE table
MODIFY (column datatype [DEFAULT expr]
[, column datatype]...) ;
```

Dropping a Table

```
DROP TABLE table ;
```

Changing the Name of an Object

```
RENAME old_name TO new_name ;
```

Truncating a Table

```
TRUNCATE TABLE table ;
```

Adding Comments to a Table

```
COMMENT ON TABLE table | COLUMN table.column
```

```

IS 'text' ;

Dropping a comment from a table
COMMENT ON TABLE table | COLUMN table.column IS '' ;

Data Dictionary
ALL_OBJECTS          USER_OBJECTS
ALL_TABLES           USER_TABLES
ALL_CATALOG          USER_CATALOG or CAT
ALL_COL_COMMENTS     USER_COL_COMMENTS
ALL_TAB_COMMENTS     USER_TAB_COMMENTS

Defining Constraints
CREATE TABLE [schema.]table
    (column datatype [DEFAULT expr] [NOT NULL]
    [column_constraint],...
    [table_constraint][,...]) ;

Column constraint level
column [CONSTRAINT constraint_name] constraint_type,
Constraint_type
PRIMARY KEY REFERENCES table(column)          UNIQUE
CHECK (condition)
Table constraint level(except NOT NULL)
column,...,[CONSTRAINT constraint_name]
    constraint_type (column,...),
NOT NULL Constraint (Only Column Level)
CONSTRAINT table[_column..._nn NOT NULL ...
UNIQUE Key Constraint
CONSTRAINT table[_column..._uk UNIQUE (column[,...])
PRIMARY Key Constraint
CONSTRAINT table[_column..._pk PRIMARY (column[,...])
FOREIGN Key Constraint
CONSTRAINT table[_column..._fk
    FOREIGN KEY (column[,...])
    REFERENCES table (column[,...])[ON DELETE CASCADE]
CHECK constraint
CONSTRAINT table[_column..._ck CHECK (condition)
Adding a NOT NULL constraint
ALTER TABLE table
    ADD [CONSTRAINT constraint_name ] type (column) ;
Adding a NOT NULL constraint
ALTER TABLE table
    MODIFY (column datatype [DEFAULT expr]
    [CONSTRAINT constraint_name_nn] NOT NULL) ;

Dropping a Constraint
ALTER TABLE table
    DROP CONSTRAINT constraint_name ;
ALTER TABLE table
    DROP PRIMARY KEY | UNIQUE (column) |
    CONSTRAINT constraint_name [CASCADE] ;

Disabling Constraints
ALTER TABLE table
    DISABLE CONSTRAINT constraint_name [CASCADE] ;
Enabling Constraints
ALTER TABLE table
    ENABLE CONSTRAINT constraint_name ;

Data Dictionary
ALL_CONSTRAINTS      USER_CONSTRAINTS
ALL_CONS_COLUMNS     USER_CONS_COLUMNS

Creating a View
CREATE [OR REPLACE] [FORCE|NOFORCE] VIEW view

    [(alias[, alias]...)]
    AS subquery
    [WITH CHECK OPTION [CONSTRAINT constraint_name]]
    [WITH READ ONLY] ;

Removing a View
DROP VIEW view ;

CREATE SEQUENCE Statement
CREATE SEQUENCE sequence
    [INCREMENT BY n]
    [START WITH n]
    [{MAXVALUE n| NOMAXVALUE}]
    [{MINVALUE n| NOMINVALUE}]
    [{CYCLE | NOCYCLE}]
    [{CACHE [n|20] | NOCACHE}] ;

Pseudocolumns
sequence.NEXTVAL      sequence.CURRVAL

Modifying a Sequence (No START WITH option)
ALTER SEQUENCE sequence
    [INCREMENT BY n]
    [{MAXVALUE n| NOMAXVALUE}]
    [{MINVALUE n| NOMINVALUE}]
    [{CYCLE | NOCYCLE}]
    [{CACHE [n|20] | NOCACHE}] ;

Removing a Sequence
DROP SEQUENCE sequence ;

Creating an Index
CREATE INDEX index
    ON TABLE (column[,column]...) ;

Removing an Index
DROP INDEX index ;

Synonyms
CREATE [PUBLIC] SYNONYM synonym FOR object ;

Removing Synonyms
DROP SYNONYM synonym ;

Data Dictionary
ALL_VIEWS            USER_VIEWS
ALL_SEQUENCES        USER_SEQUENCES
ALL_INDEXES          USER_INDEXES
ALL_IND_COLUMNS      USER_IND_COLUMNS

System Privileges (DBA)
CREATE USER           CREATE SESSION
DROP USER             CREATE TABLE
DROP ANY TABLE       CREATE SEQUENCE
BACKUP ANY TABLE     CREATE VIEW
CREATE PROCEDURE

User System Privileges

Creating Users
CREATE USER user
    IDENTIFIED BY password ;

Creating Roles
CREATE ROLE role ;

Granting System Privileges
GRANT privileges[,...] TO user[,...] ;
GRANT privileges TO role ;
GRANT role TO user[,...] ;

Changing Password
ALTER USER user IDENTIFIED BY password ;

Dropping Users
DROP USER user [CASCADE] ;

```

Dropping Roles
DROP ROLE role ;

Object Privileges

Object	Table	View	Sequence	Procedure
ALTER	X		X	
DELETE	X	X		
EXECUTE				X
INDEX	X			
INSERT	X	X		
REFERENCES	X			
SELECT	X	X	X	
UPDATE	X	X		

Object Privileges

```
GRANT object_priv [(column)]
ON object
TO {user|role|PUBLIC}
[WITH GRANT OPTION] ;
```

Revoking Object Privileges

```
REVOKE {privilege [,privilege...] | ALL}
ON object
FROM {user[,user...]|role|PUBLIC}
[CASCADE CONSTRAINTS] ;
```

Data Dictionary

Dictionary View	Dictionary View
ROLE_SYS_PRIVS	
ROLE_TAB_PRIVS	USER_ROLE_PRIVS
USER_TAB_PRIVS_MADE	USER_TAB_PRIVS_RECD
USER_COL_PRIVS_MADE	USER_COL_PRIVS_RECD

PL/SQL Block Structure

```
DECLARE --Optional
--Variables, Cursors, User-defined exceptions
BEGIN --Mandatory
--SQL statements
--PL/SQL statements
EXCEPTION --Optional
--Actions to perform when errors occur
END ; --Mandatory
```

PL/SQL Block Type

Anonymous	Procedure	Function
[DECLARE]	PROCEDURE name	FUNCTION name
	IS	RETURN datatype IS
	[DECLARE]	[DECLARE]
BEGIN	BEGIN	BEGIN
--statements	--statements	--statements
[EXCEPTION]	[EXCEPTION]	[EXCEPTION]
END ;	END ;	END ;

Declaring PL/SQL Variables

```
identifier [CONSTANT] datatype [NOT NULL]
[:=|DEFAULT expr] ;
```

Assigning Values to Variables

```
identifier := expr ;
```

Base Scalar Datatypes

Base Scalar Datatype	Base Scalar Datatype	Base Scalar Datatype	Base Scalar Datatype
VARCHAR2 (n)	NUMBER (p,s)	DATE	CHAR (n)
LONG	LONG RAW	BOOLEAN	
BINARY_INTEGER	PLS_INTEGER		

The %TYPE Attribute

```
table_name.column_name%TYPE ;
variable_name%TYPE ;
```

Composite Datatypes

Table	Record	Nested Table	Varray
LOB Datatypes			
CLOB	BLOB	BFILE	NCLOB

Creating Bind Variables

```
VARIABLE variable_name datatype
```

Displaying Bind Variables

```
PRINT [variable_name]
```

Commenting Code

```
--prefix single-line comments with two dashes
/* Place multi-line comment between the symbols */
```

SELECT Statements in PL/SQL

```
SELECT {column_list|*}
INTO {variable_name[,variable_name]...
[record_name]}
```

```
FROM table
```

```
WHERE condition
```

Implicit Cursor Attributes for DML statements

```
SQL%ROWCOUNT
```

```
SQL%FOUND
```

```
SQL%NOTFOUND
```

```
SQL%ISOPEN
```

Control Structures

IF Statement

```
IF condition THEN
statements ;
[ELSIF condition THEN
statements ;]
[ELSE
statements ;]
END IF ;
```

FOR Loop

```
FOR counter in [REVERSE]
lower..upper LOOP
statement1;
statement2;
...
END LOOP;
```

Basic Loop

```
LOOP
statements;
...
EXIT [WHEN condition];
END LOOP
```

WHILE Loop

```
WHILE condition LOOP
statement1;
statement2;
...
END LOOP ;
```

Creating a PL/SQL Record

```
TYPE record_name_type IS RECORD
(field_declaration[,field_declaration]...) ;
record_name record_name_type ;
```

Where field_declaration is

```
field_name {field_type|variable%TYPE|
table.column%TYPE|table%ROWTYPE}
[[NOT NULL] {:=|DEFAULT} expr]
```

Referencing Fields in the Record

```
record_name.field_name
```

Declaring Records with the %ROWTYPE Attribute

```
DECLARE
record_name reference%ROWTYPE
```

Creating a PL/SQL Table

```
TYPE type_name IS TABLE OF
{column_scalar_type|variable%TYPE|table.column%TYPE
|variable%ROWTYPE} [NOT NULL]
[INDEX BY BINARY_INTEGER];
identifier type_name ;
```

Referencing a PL/SQL table

```
pl_sql_table_name(primary_key_value)
```

```

Using PL/SQL Table Method
table_name.method_name[(parameters)]

PL/SQL Table Methods
EXITS(n)          COUNT  FIRST  LAST  PRIOR(n)
NEXT(n)           EXTEND(n,i)  TRIM  DELETE

PL/SQL Table of Records
TYPE table_name_type IS TABLE OF table_name%ROWTYPE
    INDEX BY BINARY_INTEGER ;
table_name table_name_type ;

Referencing a Table of Records
table_name(index).field

Declaring the Cursor in Declaration Section
CURSOR cursor_name IS select_statement ;
record_name cursor_name%ROWTYPE ;

Opening and Closing the Cursor
OPEN cursor_name ;
CLOSE cursor_name ;

Fetching Data from the Cursor
FETCH cursor_name
INTO [variable1(,variable2,...)
    |record_name] ;

Explicit Cursor Attributes
cursor_name%ISOPEN
cursor_name%NOTFOUND
cursor_name%FOUND
cursor_name%ROWCOUNT

Cursor FOR Loops
FOR record_name IN cursor_name LOOP
    statement1;
    statement2;
    ...
END LOOP;

Cursor FOR Loops Using Subqueries
FOR record_name IN (subqueries) LOOP
    statement1
    ...
END LOOP ;

Cursors with Parameters
CURSOR cursor_name [(cursor_parameter_name datatype
[,...])]
IS select_statement
[FOR UPDATE [OF column_reference] [NOWAIT]];

Parameter Name
cursor_parameter_name [IN] datatype [{:=|DEFAULT}expr]

Opening with Parameters
OPEN cursor_name(cursor_parameter_name[,...]);

Cursor FOR Loops with parameters
FOR record_name IN cursor_name(cursor_parameter_name
[,...]) LOOP
    statement1;
    statement2;
    ...
END LOOP;

WHERE CURRENT OF clause
UPDATE|DELETE ... WHERE CURRENT OF cursor_name ;

Predefined Exceptions
NO_DATA_FOUND
TOO_MANY_ROWS

INVALID_CURSOR
ZERO_DIVIDE
DUP_VAL_ON_INDEX

Trapping Exceptions
EXCEPTION
    WHEN exception1 [OR exception2 ...] THEN
        statement1 ;
        statement2 ;
        ...
    [WHEN exception3 [OR exception4 ...] THEN
        statement1 ;
        statement2 ;
        ...]
    [WHEN OTHERS THEN
        statement1 ;
        statement2 ;
        ...]

Declaring Non-Predefined Oracle Sever Exception
DECLARE
    exception EXCEPTION ;
    PRAGMA EXCEPTION_INIT(exception, error_number) ;

Referencing the declared Non-predefined exception
BEGIN
    ...
EXCEPTION
    WHEN exception THEN
        statement1 ;
        ...
END ;

Trapping User-Defined Exceptions
DECLARE
    exception EXCEPTION ;
BEGIN
    ...
    IF SQL%NOTFOUND THEN
        RAISE exception ;
    END IF ;
    ...
EXCEPTION
    WHEN exception THEN
        statement1 ;
        ...
END ;

Functions for Trapping Exceptions
SQLCODE          return error code
SQLERRM          return error message
RAISE_APPLICATION_ERROR procedure(Executable/Exception
Section)
RAISE_APPLICATION_ERROR ( error_number,
    message [, {TRUE|FALSE}]) ;
error_number      between -20000 to -20999
message           string up to 2,048 bytes long
TRUE              placed on the stack of previous errors.
FALSE             replaces all previous errors

Single-Row Functions
Character Functions
LOWER(column|expression)
UPPER(column|expression)

```

```

INITCAP(column|expression)
INSTR(column|expression,m)
CONCAT(column1|expression1,column2|expression2)
SUBSTR(column|expression,m,[n])
LENGTH(column|expression)
LPAD(column|expression,n,'string')
Number Functions
MOD(m,n)
ROUND(column|expression,n)
TRUNC(column|expression,n)
Date Functions
MONTHS_BETWEEN(date1,date2)
ADD_MONTHS(date,n)
NEXT_DAY(date,'char')
LAST_DAY(date)
ROUND(date,['fmt'])
TRUNC(date,['fmt'])
Conversion Functions
TO_CHAR(number|date,['fmt'])
TO_NUMBER(char,['fmt'])
TO_DATE(char,['fmt'])
NVL(expr1,expr2)
DECODE(col/expr,search1,result1
      [,search2,result2,...]
      [,default])

Operators
Comparison      =  >  >=  <  <=  <>
                BETWEEN..AND, IN, LIKE, IS NULL
Logical         AND      OR      NOT
Order of Operations
Operator        Operation
**,NOT          Exponentiation, logical negation
+,-             Identity, negation
*,/             Multiplication, division
+,-,||          Addition, subtraction, concatenation
=,!=,<,>,<=     Comparison
>=,IS NULL,LIKE
BETWEEN,IN
AND             Conjunction
OR              Inclusion

```

Two Unique Languages

Chapter 9 - PL/SQL

Procedural Language Extension of SQL
